

## 第 11 章

## 线性表实验

## CHAPTER

线性表是最简单的基本数据结构,在实际问题中有着广泛的应用。通过本章的验证实验,巩固对线性表逻辑结构的理解,掌握线性表的存储结构及基本操作的实现,为应用线性表解决实际问题奠定良好的基础。通过本章的设计实验和综合实验,进一步培养以线性表作为数据结构解决实际问题的应用能力。

## 11.1 验证实验

### 11.1.1 顺序表的实现

#### 1. 实验目的

- (1) 掌握线性表的顺序存储结构;
- (2) 验证顺序表及其基本操作的实现;
- (3) 理解算法与程序的关系,能够将顺序表算法转换为对应的程序。

#### 2. 实验内容

- (1) 建立含有若干个元素的顺序表;
- (2) 对已建立的顺序表实现插入、删除、查找等基本操作。

#### 3. 实现提示

定义顺序表的数据类型——顺序表类 SeqList,包括题目要求的插入、删除、查找等基本操作,为便于查看操作结果,设计一个输出函数依次输出顺序表的元素。顺序表类 SeqList 的定义以及基本操作的算法请参见主教材 2.2 节。

简单起见,本实验假定线性表的数据元素为 int 型,要求学生:

- (1) 将实验程序调试通过后,用模板类改写;
- (2) 加入求线性表的长度等基本操作;
- (3) 重新给定测试数据,验证抛出异常机制。

#### 4. 实验程序

在 VC++ 编程环境下新建一个工程“顺序表验证实验”,在该工程中新

建一个头文件 SeqList.h, 该头文件包括顺序表类 SeqList 的定义, 范例程序如下:

```
#ifndef SeqList_H                                //避免重复包含 SeqList.h 头文件
#define SeqList_H
const int MaxSize = 10;                          //线性表最多有 10 个元素
class SeqList
{
public:
    SeqList(){length = 0;}                        //无参构造函数, 创建一个空表
    SeqList(int a[], int n);                      //有参构造函数
    ~SeqList(){}                                 //析构造函数
    void Insert(int i, int x);                    //在线性表第 i 个位置插入值为 x 的元素
    int Delete(int i);                           //删除线性表的第 i 个元素
    int Locate(int x);                           //求线性表中值为 x 的元素序号
    void PrintList();                            //按序号依次输出各元素
private:
    int data[MaxSize];                          //存放数据元素的数组
    int length;                                 //线性表的长度
};
#endif
```

在工程“顺序表验证实验”中新建一个源程序文件 SeqList.cpp, 该文件包括类 SeqList 中成员函数的定义, 范例程序如下:

```
#include <iostream>                               //引入输入输出流
using namespace std;
#include "SeqList.h"                              //引入类 SeqList 的声明
//以下是类 SeqList 的成员函数定义

SeqList::SeqList(int a[], int n)
{
    if (n > MaxSize) throw "参数非法";
    for (int i = 0; i < n; i++)
        data[i] = a[i];
    length = n;
}

void SeqList::Insert(int i, int x)
{
    if (length >= MaxSize) throw "上溢";
    if (i < 1 || i > length + 1) throw "位置非法";
    for (int j = length; j >= i; j--)
        data[j] = data[j - 1];                  //第 j 个元素存在数组下标为 j - 1 处
    data[i - 1] = x;
    length++;
}
```

```

int SeqList::Delete(int i)
{
    if (length == 0) throw "下溢";
    if (i < 1 || i > length) throw "位置非法";
    int x = data[i - 1];
    for (int j = i; j < length; j++)
        data[j - 1] = data[j];           //此处 j 已经是元素所在的数组下标
    length--;
    return x;
}

int SeqList::Locate(int x)
{
    for (int i = 0; i < length; i++)
        if (data[i] == x) return i + 1; //下标为 i 的元素其序号为 i + 1
    return 0;                          //退出循环,说明查找失败
}

void SeqList::PrintList()
{
    for (int i = 0; i < length; i++)
        cout << data[i] << " ";
    cout << endl;
}

```

在工程“顺序表验证实验”中新建一个源程序文件 SeqList\_main.cpp, 该文件包括主函数, 范例程序如下:

```

#include <iostream>           //引入输入输出流
using namespace std;
#include "SeqList.h"         //引入类 SeqList 的声明

void main()
{
    int r[5] = {1, 2, 3, 4, 5};
    SeqList L(r, 5);
    cout << "执行插入操作前数据为:" << endl;
    L.PrintList();           //输出所有元素
    try
    {
        L.Insert(2, 3);      //在第 2 个位置插入值为 3 的元素
    }
    catch (char * s)
    {
        cout << s << endl;
    }
}

```

```
}  
cout<<"执行插入操作后数据为:"<<endl;  
L.PrintList(); //输出所有元素  
cout<<"值为3的元素位置为:";  
cout<<L.Locate(3)<<endl; //查找元素3,并返回在顺序表中位置  
cout<<"执行删除第一个元素操作,删除前数据为:"<<endl;  
L.PrintList(); //输出所有元素  
try  
{  
    L.Delete(1); //删除第1个元素  
}  
catch (char * s)  
{  
    cout<<s<<endl;  
}  
cout<<"删除后数据为:"<<endl;  
L.PrintList(); //输出所有元素  
}
```

### 11.1.2 单链表的实现

#### 1. 实验目的

- (1) 掌握线性表的链接存储结构;
- (2) 验证单链表及其基本操作的实现;
- (3) 进一步理解算法与程序的关系,能够将单链表算法转换为对应的程序。

#### 2. 实验内容

- (1) 用头插法(或尾插法)建立带头结点的单链表;
- (2) 对已建立的单链表实现插入、删除、查找等基本操作。

#### 3. 实现提示

定义单链表的结点结构,定义单链表的数据类型——单链表类 LinkList,包括题目要求的插入、删除、查找等基本操作,为便于查看操作结果,设计一个输出函数依次输出单链表的元素。单链表的结点结构、单链表类 LinkList 的定义以及基本操作的算法请参见主教材 2.3.1 节。

有了顺序表的实验基础,本实验采用模板实现,要求学生:

- (1) 体会加入模板后的程序有什么变化;
- (2) 加入求线性表的长度等基本操作;
- (3) 重新给定测试数据,验证抛出异常机制;
- (4) 将单链表的结点结构用结点类实现。

#### 4. 实验程序

在 VC++ 编程环境下新建一个工程“单链表验证实验”,在该工程中新建一个头文件



LinkedList.h, 该头文件包括单链表类 LinkedList 的定义, 范例程序如下:

```
#ifndef LinkedList_H                                //避免重复包含 LinkedList.h 头文件
#define LinkedList_H

//以下定义单链表的结点

template <class DataType>
struct Node
{
    DataType data;
    Node<DataType> * next;
};

//以下是类 LinkedList 的声明

template <class DataType>
class LinkedList
{
public:
    LinkedList();                                //建立只有头结点的空链表
    LinkedList(DataType a[], int n);            //建立有 n 个元素的单链表
    ~LinkedList();                                //析构函数
    int Locate(DataType x);                      //在单链表中查找值为 x 的元素序号
    void Insert(int i, DataType x);              //在第 i 个位置插入元素值为 x 的结点
    DataType Delete(int i);                     //在单链表中删除第 i 个结点
    void PrintList();                            //按序号依次输出各元素
private:
    Node<DataType> * first;                      //单链表的头指针
};
#endif
```

在工程“单链表验证实验”中新建一个源程序文件 LinkedList.cpp, 该文件包括类 LinkedList 中成员函数的定义, 范例程序如下:

```
#include <iostream>                                //引入输入输出流
using namespace std;
#include "LinkedList.h"                            //引入类 LinkedList 的声明
//以下为类 LinkedList 的成员函数定义

template <class DataType>
LinkedList<DataType>::LinkedList()
{
    first = new Node<DataType>;                  //生成头结点
    first->next = NULL;                          //头结点的指针域置空
}

template <class DataType>
LinkedList<DataType>::LinkedList(DataType a[], int n)
{
    Node<DataType> * r, * s;
    first = new Node<DataType>;                  //生成头结点
```

```

    r = first;                                     //尾指针初始化
    for (int i = 0; i < n; i++)
    {
        s = new Node<DataType>;
        s->data = a[i];                           //为每个数组元素建立一个结点
        r->next = s; r = s;                       //将结点 s 插入到终端结点之后
    }
    r->next = NULL;                               //将终端结点的指针域置空
}

template <class DataType>
LinkedList<DataType>::~~LinkedList()
{
    Node<DataType> * q = NULL;
    while (first != NULL)                         //释放单链表的每一个结点的存储空间
    {
        q = first;                                //暂存被释放结点
        first = first->next;                       //first 指向被释放结点的下一个结点
        delete q;
    }
}

template <class DataType>
void LinkedList<DataType>::Insert(int i, DataType x)
{
    Node<DataType> * p = first, * s = NULL;       //工作指针 p 指向头结点
    int count = 0;
    while (p != NULL && count < i - 1)           //查找第 i-1 个结点
    {
        p = p->next;                               //工作指针 p 后移
        count++;
    }
    if (p == NULL) throw "位置";                 //没有找到第 i-1 个结点
    else {
        s = new Node<DataType>; s->data = x;      //结点 s 的数据域为 x
        s->next = p->next; p->next = s;           //将结点 s 插入到结点 p 之后
    }
}

template <class DataType>
DataType LinkedList<DataType>::Delete(int i)
{
    Node<DataType> * p = first, * q = NULL;       //工作指针 p 指向头结点
    DataType x;
    int count = 0;
    while (p != NULL && count < i - 1)           //查找第 i-1 个结点

```

```

{
    p = p->next;
    count++;
}
if (p == NULL || p->next == NULL)           //结点 p 或 p 的后继结点不存在
    throw "位置";
else {
    q = p->next; x = q->data;                 //暂存被删结点
    p->next = q->next;                       //摘链
    delete q;
    return x;
}
}

template <class DataType>
int LinkedList<DataType>::Locate(DataType x)
{
    Node<DataType> * p = first->next;        //工作指针 p 初始化
    int count = 1;                          //累加器 count 初始化
    while (p != NULL)
    {
        if (p->data == x) return count;      //查找成功,返回序号
        p = p->next;
        count++;
    }
    return 0;                               //退出循环表明查找失败
}

template <class DataType>
void LinkedList<DataType>::PrintList()
{
    Node<DataType> * p = first->next;        //工作指针 p 初始化
    while (p != NULL)
    {
        cout<<p->data<<" ";
        p = p->next;                        //工作指针 p 后移
    }
    cout<<endl;
}

```

在工程“单链表验证实验”中新建一个源程序文件 `LinkedList_main.cpp`, 该文件包括主函数, 范例程序如下:

```

#include <iostream>                          //引入输入输出流
using namespace std;
#include "LinkedList.cpp"                    //引入类 LinkedList 的成员函数定义

```

```

//以下为主函数

void main()
{
    int r[5] = {1, 2, 3, 4, 5};
    LinkList<int> L(r, 5);
    cout<<"执行插入操作前数据为:"<<endl;
    L.PrintList(); //显示链表中所有元素
    try
    {
        L.Insert(2, 3); //在第2个位置插入值为3的元素
    }
    catch (char * s)
    {
        cout<<s<<endl;
    }
    cout<<"执行插入操作后数据为:"<<endl;
    L.PrintList(); //显示单链表的所有元素
    cout<<"值为5的元素位置为:";
    cout<<L.Locate(5)<<endl; //查找元素5,并返回在单链表中位置
    cout<<"执行删除操作前数据为:"<<endl;
    L.PrintList(); //显示单链表的所有元素
    try
    {
        L.Delete(1); //删除第1个元素
    }
    catch (char * s)
    {
        cout<<s<<endl;
    }
    cout<<"执行删除操作后数据为:"<<endl;
    L.PrintList(); //显示单链表的所有元素
}

```

## 11.2 设计实验

### 11.2.1 约瑟夫环问题

#### 1. 问题描述

设有编号为  $1, 2, \dots, n$  的  $n$  ( $n > 0$ ) 个人围成一个圈, 每个人持有一个密码  $m$ , 从第 1 个人开始报数, 报到  $m$  时停止报数, 报  $m$  的人出圈, 再从他的下一个人起重新报数, 报到  $m$  时停止报数, 报  $m$  的出圈,  $\dots$ , 直到所有人全部出圈为止。当任意给定  $n$  和  $m$  后, 求  $n$  个人出圈的次序。

#### 2. 基本要求

(1) 建立数据模型, 确定存储结构;



- (2) 对任意  $n$  个人, 密码为  $m$ , 实现约瑟夫环问题;  
 (3) 出圈的顺序可以依次输出, 也可以用数组存储。

### 3. 设计思想

约瑟夫环问题的存储结构。由于约瑟夫环问题本身具有循环性质, 考虑采用循环链表, 为了统一对表中任意结点的操作, 循环链表不带头结点。将循环链表的结点定义为如下结构类型:

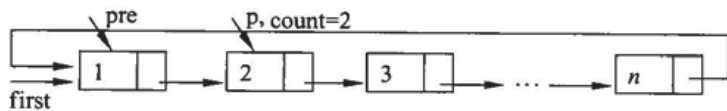
```
struct Node
{
    int data;           //编号
    Node * next;
};
```

建立约瑟夫环。建立一个不带头结点的循环链表并由头指针  $first$  指示, 如图 11-1(a) 所示, 具体算法与建立单链表类似。

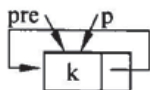
设计约瑟夫环算法实现出圈。下面给出算法的伪代码描述, 操作示意图如图 11-1 所示。

**伪代码**

1. 工作指针  $pre$  和  $p$  初始化, 计数器  $count$  初始化;  
 $pre=first; p=first \rightarrow next; count=2;$
2. 循环直到  $p$  等于  $pre$ 
  - 2.1 如果  $count$  等于  $m$ , 则
    - 2.1.1 输出结点  $p$  的编号;
    - 2.1.2 删除结点  $p; p=pre \rightarrow next;$
    - 2.1.3 计数器  $count$  重新开始计数;
  - 2.2 否则, 执行
    - 2.2.1 工作指针  $pre$  和  $p$  后移;
    - 2.2.2 计数器  $count$  增 1;
3. 退出循环, 链表中只剩下一个结点  $p$ , 输出结点  $p$  后将结点  $p$  删除;



(a) 建立约瑟夫环



(b) 循环结束条件

图 11-1 约瑟夫环问题存储示意图

### 4. 思考题

- (1) 采用顺序存储结构如何实现约瑟夫环问题?  
 (2) 可以建立一个由尾指针  $rear$  指示的不带头结点的循环链表, 则初始时就可以从

1 开始计数,实现这个算法。

(3) 如果每个人持有的密码不同,应如何实现约瑟夫环问题?

## 11.2.2 用单链表实现集合的操作

### 1. 问题描述

用有序单链表实现集合的判等、交、并和差等基本运算。

### 2. 基本要求

- (1) 对集合中的元素用有序单链表进行存储;
- (2) 实现交、并、差等基本运算时,不能另外申请存储空间;
- (3) 充分利用单链表的有序性,要求算法有较好的时间性能。

### 3. 设计思想

集合是由互不相同的元素构成的一个整体,在集合中,元素之间可以没有任何关系,所以,集合也可作为线性表处理。用单链表实现集合的操作,需要注意集合中元素的唯一性,即在单链表中不存在值相同的结点。本实验要求采用有序单链表,还要注意利用单链表的有序性。

(1) 判断  $A$  和  $B$  是否相等。两个集合相等的条件是不仅长度相同,而且各个对应的元素也相等。由于用有序单链表表示集合,所以只要同步扫描两个单链表,若从头至尾每个对应的元素都相等,则表明两个集合相等。具体算法如下:

判断集合相等算法 IsEqual

```
int IsEqual(Node *A, Node *B)           //A 和 B 分别是两个单链表的头指针
{
    pa = A->next; pb = B->next;
    while (pa != NULL && pb != NULL)
    {
        if (pa->data == pb->data) {
            pa = pa->next;
            pb = pb->next;
        }
        else break;
    }
    if (pa == NULL && pb == NULL) return 1;
    else return 0;
}
```

(2) 求集合  $A$  和  $B$  的交集。根据集合的运算规则,集合  $A \cap B$  中包含所有既属于集合  $A$  又属于集合  $B$  的元素,因此,需查找单链表  $A$  和  $B$  中的相同元素并保留在单链表  $A$  中。由于用有序单链表表示集合,因此判断某元素是否在  $B$  中不需要遍历表  $B$ ,而是从上次搜索到的位置开始,若在搜索过程中,遇到一个其值比该元素大的结点,便可断定该元素不在单链表中,为此,需用两个指针  $p$ 、 $q$  分别指向当前被比较的两个结点,会出现以



下三种情况:

- ① 若  $p \rightarrow data > q \rightarrow data$ , 说明还未找到, 需在表  $B$  中继续查找;
- ② 若  $p \rightarrow data < q \rightarrow data$ , 说明表  $B$  中无此值, 处理表  $A$  中下一结点;
- ③ 若  $p \rightarrow data = q \rightarrow data$ , 说明找到了公共元素。

具体算法如下:

#### 求集合的交集算法 Interest

```
void Interest(Node * A, Node * B)
{
    //A、B 分别是两个单链表的头指针, 最后的结果在单链表 A 中
    pre = A; p = A -> next; q = B -> next;
    while (p != NULL && q != NULL)
    {
        if (p -> data < q -> data) {
            pre -> next = p -> next;
            delete p;
            p = pre -> next;
        }
        else if (p -> data > q -> data) q = q -> next;
        else {
            p = p -> next;
            q = q -> next;
        }
    }
}
```

(3) 求集合  $A$  和  $B$  的并集。根据集合的运算规则, 集合  $A \cup B$  中包含所有或属于集合  $A$  或属于集合  $B$  的元素。因此, 对单链表  $B$  中的每个元素  $x$ , 在单链表  $A$  中进行查找, 若存在和  $x$  不相同的元素, 则将该结点插入到单链表  $A$  中。算法请参照求集合的交集自行设计。

(4) 求集合  $A$  和  $B$  的差集。根据集合的运算规则, 集合  $A - B$  中包含所有属于集合  $A$  而不属于集合  $B$  的元素。因此, 对单链表  $B$  中的每个元素  $x$ , 在单链表  $A$  中进行查找, 若存在和  $x$  相同的结点, 则将该结点从单链表  $A$  中删除。算法请参照求集合的交集自行设计。

在主函数中, 首先建立两个有序单链表表示集合  $A$  和  $B$ , 然后依次调用相应函数实现集合的判等、交、并和差等运算, 并输出运算结果。单链表的结点结构和建立算法请参见主教材 2.3.1 节。

**注意:** 利用头插法建立有序单链表, 实参数组应该是降序排列, 利用尾插法建立有序单链表, 实参数组应该是升序排列。

#### 4. 思考题

- (1) 如果要求将交、并、差等运算的结果保存在一个新的有序单链表中, 应如何修改算法?
- (2) 如果表示集合的单链表是无序的, 应如何实现集合的判等、交、并和差等基本运

算? 时间性能如何?

## 11.3 综合实验

### 11.3.1 大整数的代数运算

#### 1. 问题描述

C/C++ 语言中的 int 类型能表示的整数范围是  $-2^{31} \sim 2^{31} - 1$ , unsigned int 类型能表示的整数范围是  $0 \sim 2^{32} - 1$ , 即  $0 \sim 4\,294\,967\,295$ , 所以, int 和 unsigned int 类型都不能存储超过 10 位的整数。有些问题需要处理的整数远远不止 10 位, 这种大整数用 C/C++ 语言的基本数据类型无法直接表示。请编写算法完成两个大整数的加、减、乘和除等基本的代数运算。

#### 2. 基本要求

- (1) 大整数的长度在 100 位以下;
- (2) 设计存储结构表示大整数;
- (3) 设计算法实现两个大整数的加、减、乘和除等基本的代数运算;
- (4) 分析算法的时间复杂度和空间复杂度。

#### 3. 设计思想

处理大整数的一般方法是用数组存储大整数, 即开一个比较大的整型数组, 数组元素代表大整数的一位, 通过数组元素的运算模拟大整数的运算。根据计算的方便性决定将大整数由低位到高位还是由高位到低位存储到数组中, 例如乘法是由低位到高位进行运算, 并且可能要向高位产生进位, 所以应该由低位到高位存储。如果从键盘输入大整数, 一般用字符数组存储, 这样无需对大整数进行分段输入, 当然输入到字符数组后需要将字符转换为数字。下面讨论大整数的加法运算, 其他操作请读者自行完成。

已知大整数  $A = a_1 a_2 \cdots a_n$ ,  $B = b_1 b_2 \cdots b_m$ , 求  $C = A + B$ 。可以用两个顺序表 A 和 B 分别存储两个大整数, 用顺序表 C 存储求和的结果。为了便于执行加法运算, 可以将大整数的低位存储到顺序表的低端, 顺序表的长度表示大整数的位数。图 11-2 给出了大整数求和的操作示意图, 算法的伪代码描述如下:

伪代码

1. 初始化进位标志 flag = 0;
2. 求大整数 A 和 B 的长度:  $n = A.length$ ;  $m = B.length$ ;
3. 从个位开始逐位进行第 i 位的加法, 直到 A 或 B 计算完毕
  - 3.1 计算第 i 位的值:  $C.data[i] = (A.data[i] + B.data[i] + flag) \% 10$ ;
  - 3.2 计算该位的进位:  $flag = (A.data[i] + B.data[i] + flag) / 10$ ;
4. 计算大整数 A 或 B 余下的部分;
5. 计算结果的位数;



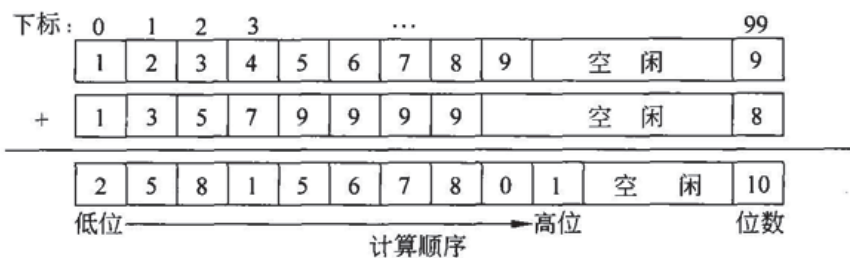


图 11-2 大整数加法的操作过程(个位存储在数组下标为 0 的数组单元)

### 11.3.2 一元多项式相加

#### 1. 问题描述

已知  $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$  和  $B(x) = b_0 + b_1x + b_2x^2 + \dots + b_mx^m$ , 并且在  $A(x)$  和  $B(x)$  中指数相差很多, 求  $A(x) + B(x)$ 。

#### 2. 基本要求

- (1) 设计存储结构表示一元多项式;
- (2) 设计算法实现一元多项式相加;
- (3) 分析算法的时间复杂度和空间复杂度。

#### 3. 设计思想

一元多项式求和实质上是合并同类项的过程, 其运算规则为: (1) 若两项的指数相等, 则系数相加; (2) 若两项的指数不等, 则将两项加在结果中。

一元多项式  $A(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n$  由  $n+1$  个系数唯一确定, 因此, 可以用一个线性表  $(a_0, a_1, a_2, \dots, a_n)$  来表示, 每一项的指数  $i$  隐含在其系数  $a_i$  的序号里。但是, 当多项式的指数很高且变化很大时, 在表示多项式的线性表中就会存在很多 0 元素。一个较好的存储方法是只存非 0 元素, 但是需要在存储非 0 元素系数的同时存储相应的指数。这样, 一个一元多项式的每一个非 0 项可由系数和指数唯一表示。

由于两个一元多项式相加后, 会改变多项式的系数和指数, 因此采用顺序表不合适。采用单链表存储, 则每一个非 0 项对应单链表中的一个结点, 且单链表应按指数递增有序排列。结点结构如图 11-3 所示。

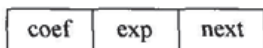


图 11-3 一元多项式单链表的结点结构

其中:

coef 是系数域, 存放非 0 项的系数;

exp 是指数域, 存放非 0 项的指数;

next 是指针域, 存放指向下一结点的指针。

将两个一元多项式用两个单链表存储后, 如何实现二者相加呢?

设两个工作指针  $p$  和  $q$ , 分别指向两个单链表的开始结点。通过对结点  $p$  的指数域



和结点  $q$  的指数域进行比较进行同类项合并,则出现下列三种情况:

- (1) 若  $p \rightarrow \text{exp}$  小于  $q \rightarrow \text{exp}$ , 则结点  $p$  应为结果中的一个结点。
- (2) 若  $p \rightarrow \text{exp}$  大于  $q \rightarrow \text{exp}$ , 则结点  $q$  应为结果中的一个结点, 将  $q$  插入到第一个链表中结点  $p$  之前。
- (3) 若  $p \rightarrow \text{exp}$  等于  $q \rightarrow \text{exp}$ , 则结点  $p$  与结点  $q$  为同类项, 将  $q$  的系数加到  $p$  的系数上。若相加结果不为 0, 则结点  $p$  应为结果中的一个结点, 同时删除结点  $q$ ; 若相加结果为 0, 则表明结果中无此项, 删除结点  $p$  和结点  $q$ 。

算法的伪代码描述如下:

伪代码

1. 初始化工作指针  $p$  和  $q$ ;
2. while ( $p$  和  $q$  非空) 执行下列三种情形之一:
  - 2.1 如果  $p \rightarrow \text{exp}$  小于  $q \rightarrow \text{exp}$ , 则指针  $p$  后移;
  - 2.2 如果  $p \rightarrow \text{exp}$  大于  $q \rightarrow \text{exp}$ , 则
    - 2.2.1 将结点  $q$  插入到结点  $p$  之前;
    - 2.2.2 指针  $q$  指向原指结点的下一个结点;
  - 2.3 如果  $p \rightarrow \text{exp}$  等于  $q \rightarrow \text{exp}$ , 则
    - 2.3.1  $p \rightarrow \text{coef} = p \rightarrow \text{coef} + q \rightarrow \text{coef}$ ;
    - 2.3.2 如果  $p \rightarrow \text{coef}$  等于 0, 则执行下列操作, 否则指针  $p$  后移;
      - 2.3.2.1 删除结点  $p$ ;
      - 2.3.2.2 使指针  $p$  指向它原指结点的下一个结点;
    - 2.3.3 删除结点  $q$ ;
    - 2.3.4 使指针  $q$  指向它原指结点的下一个结点;
3. 如果  $q$  不为空, 将结点  $q$  链接在第一个单链表的后面;

#### 4. 思考题

用顺序表如何实现两个多项式相加? 设计算法并与单链表实现进行比较。